

Design Patterns

Mulakat Hazirlik Notlari

Strategy · Singleton · Proxy

Acidan basla, kodla isle, SOLID ile bagla, mulakatci nereden vurursa cevabini al.

ARGUS / HR-WEB orneklerinin esliginde

1. Strategy Pattern

Aynı isin farklı yapılma şekillerini, her birini ayrı bir sınıfa koyarak yönet.

Once sezgi, sonra pattern

Pattern lafini bir dakika unut. Su sahneye bak: bir kafe actin. İlk gün sadece nakit alıyorsun, basit. İkinci ay kart geldi, POS kurdun. Altıncı ay yemek kartları (Sodexo, Multinet). Bir yıl sonra hediye çeki, kupon kodu, taksitli ödeme...

Şimdi bütün bu ödeme tiplerini **tek bir kasiyere** yaptırıyorsan, o kasiyer bir noktada aklını yitirir. Çünkü her ödeme tipinin **kendi iç mantığı** var ama **sonucu hep aynı**: müşteri ödemiş olacak.

Yazılımda da aynı: aynı sonuca giden, farklı mantıkları olan iş kuralları. Bunları tek bir sınıfa tikarsan bir süre sonra o sınıf çöker. Strategy bu çokmeyi engellemek için doğdu. Pattern'in özü tek cümle: **aynı isin farklı yapılma şekillerini, her birini ayrı bir sınıfa koyarak yönet.**

Aciyi kodda görelim

ARGUS'tan giderim. İzin onayı:

```
public class IzinOnayService {  
  
    public OnayResult onayla(Izin izin) {  
        if (izin.getTip().equals("YILLIK")) {  
            // sef onayı kontrolü  
            // birikmiş izin günü hesabi  
            // sef yoksa İK'ya yönlendir  
            return ...;  
        } else if (izin.getTip().equals("MAZERET")) {  
            // 3 güne kadar İK muduru  
            // fazlasında direktör  
            return ...;  
        } else if (izin.getTip().equals("RAPORLU")) {  
            // SGK raporu var mı kontrolü  
            // onay gerektirmez ama bildirim atılmalı  
            return ...;  
        } else if (izin.getTip().equals("DOĞUM")) {  
            // otomatik onay, sadece kayıt  
            return ...;  
        }  
        // ... 8 ay sonra 14 dâ  
    }  
}
```

Bu kod **calisiyor**. Production'da. Müşteri memnun. Sorun ne?

Sorun yarın cikiyor. "Babalık izni" geliyor — yeni else if. "Evlilik izni" geliyor — yeni else if. Yıllık iznin kuralı bir gün değişiyor, sen orayı düzenlerken yan branch'te mazeretin parantezini bozuyorsun. Üç aydır iyi çalışan bir şey, **alakasız** bir değişiklikten oturuyor.

Asil yara: birbirinden habersiz olması gereken iş kuralları, aynı binayı paylaşıyor. Yıllık iznin değişikliğinin mazeret iznine dokunma *ihtimali* bile olmamalı. Mevcut kodda bu garanti yok.

Strategy'nin yaptığı şey: izolasyon

Her if dalinin içerisini ayrı bir kutuya koyarsan, kutular birbirinden habersiz olur. Bir kutuyu aktifte diğerleri etkilenmez. Pattern bu kadar basit aslında.

Önce ortak dili kur — bütün strategy'lerin uyacağı sözleşme:

```
public interface IzinOnayStrategy {
    IzinTipi destekledigiTip();
    OnayResult onayla(Izin izin, Personel personel);
}
```

Her iş kuralı kendi evine tasiniyor:

```
@Component
public class YillikIzinOnayStrategy implements IzinOnayStrategy {
    private final PersonelRepository personelRepo;

    public IzinTipi destekledigiTip() { return IzinTipi.YILLIK; }

    @Override
    public OnayResult onayla(Izin izin, Personel personel) {
        Personel sef = personelRepo.findSef(personel);
        if (sef == null) {
            // IK'ya yönlendir
        }
        // sefe bildirim, birikmiş gün hesabı vs.
        return new OnayResult(...);
    }
}

@Component
public class MazeretIzinOnayStrategy implements IzinOnayStrategy {
    public IzinTipi destekledigiTip() { return IzinTipi.MAZERET; }

    @Override
    public OnayResult onayla(Izin izin, Personel personel) {
        if (izin.getGunSayisi() > 3) {
            // direktör onayı
        }
        // IK muduru onayı
        return new OnayResult(...);
    }
}
```

Sonra orkestrayı yöneten Context — pattern'da bu sınıfın adı budur:

```
@Service
public class IzinOnayService {
    private final Map<IzinTipi, IzinOnayStrategy> strategies;

    public IzinOnayService(List<IzinOnayStrategy> tumStrategiler) {
        // Spring tüm IzinOnayStrategy bean'lerini otomatik enjekte eder
        this.strategies = tumStrategiler.stream()
            .collect(Collectors.toMap(s -> s.destekledigiTip(), s -> s));
    }

    public OnayResult onayla(Izin izin, Personel personel) {
        IzinOnayStrategy strategy = strategies.get(izin.getTip());
    }
}
```

```

        if (strategy == null) {
            throw new BilinmeyenIzinTipiException(izin.getTip());
        }
        return strategy.onayla(izin, personel);
    }
}

```

Yarin babalik izni geldi. Ne yapiyorsun?

```

@Component
public class BabalikIzinOnayStrategy implements IzinOnayStrategy {
    public IzinTipi destekledigiTip() { return IzinTipi.BABALIK; }

    @Override
    public OnayResult onayla(Izin izin, Personel personel) {
        return new OnayResult(OnayDurum.OTOMATIK_ONAYLI, 5);
    }
}

```

Hepsi bu. IzinOnayService'i acmadin. Diger strategy'lere dokunmadin. Yeni bir kutu attin, sistem onu kendi kesfetti, kullanmaya basladi. Spring container, constructor'da List<IzinOnayStrategy>'i yeniden toplarken yeni bean'i de ekledi.

Pattern'in dört oyuncusu (mulakat terminolojisi)

GoF kitabindaki isimlerle bil, cunku mulakatci bu isimleri kullanir:

Strategy (interface) — Ortak sozlesme. IzinOnayStrategy.

ConcreteStrategy — Her bir somut implementation. YillikIzinOnayStrategy vs.

Context — Strategy'yi kullanan, hangi strategy ile calistigini disaridan alan sinif. IzinOnayService.

Client — Context'i ve strategy secimini orkestre eden taraf. Genelde controller veya ust seviye servis.

Kritik nuans: Context, strategy'yi kendi secmez. Ya client ona soyler, ya da bir registry/factory cozer. Context'in icinde if (tip == X) new XStrategy() yazarsan, sorunu cozmeyisin — sadece bir kat asagi itmisin. Bu hata cok yapilir, mulakatci bunu test eder.

SOLID baglantisi — altin madeni

Mulakatci sana "Strategy'nin SOLID ile iliskisi ne?" diye sordugunda, ezbere degil, **kodla gostererek** anlatmalisin. Aralarindaki bag yuzeysel degil, mimari.

O — Open/Closed Principle

Bertrand Meyer'in cumlesi: "Software entities should be open for extension, but closed for modification." Strategy bu prensibin **kanonik ornegi**, ders kitabi ornegi. Yeni izin tipi geldiginde mevcut hicbir sinifi **acmiyorsun**, sadece yeni bir sinif **ekliyorsun**. Bu duyduğun en saf OCP.

Mulakatta "OCP'ye somut ornek ver" derse cevap Strategy'dir. Anlatirken su cumleyi kur: "Yeni bir izin tipi geldiginde IzinOnayService'i hic acmiyorum bile, sadece yeni bir

@Component ekliyorum." O cumle OCP'yi gercekten anladigini gosterir.

S — Single Responsibility

Eski koda IzinOnayService.onayla() metodunun 14 farkli sorumlulugu vardi. Strategy bunu parampara etti — her ConcreteStrategy **tek bir is kuralinin** sahibi. YillikIzinOnayStrategy sadece yillik izin icin var, mazeretten haberi bile yok. Tek bir sebeple degisir: yillik izin kurali degisirse.

L — Liskov Substitution

"Bir base type'in yerine alt tipi koydugunda program davranisi bozulmamali." Tum IzinOnayStrategy'ler birbirinin yerine gecebiliyor. IzinOnayService hangisinin geldigini umursamiyor — sozlesmeye uydugu surece akiyor. Pattern LSP'yi **zorla** uygulatiyor, sen bilmesen bile.

I — Interface Segregation

Strategy interface'i **dar tutulmus**: bir tane gercek metot (onayla) ve bir kesif metodu (destekledigiTip). Strategy'lere kullanmayacaklari metotlar dayatilmiyor. Bir gun interface'e 8 metot koyup bazi strategy'lerin yarisini bos birakmak zorunda kalirsa, ISP'yi ihlal etmeye basliyorsun.

D — Dependency Inversion

IzinOnayService somut siniflara **degil**, interface'e bagimli. Yuksek seviye modul (Service), dusuk seviye modullere (strategy'ler) bagimli degil — ikisi de soyutlamaya bagimli. Test'te MockIzinOnayStrategy enjekte edebiliyorsun, controller'i integration test'lemek icin tum DB kurmana gerek kalmiyor.

DIP'in ince nuansi (kidemli sorusu): Interface high-level'a aittir. Cunku o, high-level'in ihtiyaclarini ifade eder. "*Bana izin onaylayabilen bir sey lazim, su metodu vermesi gerekiyor*" diyen IzinOnayService. Strategy'ler bu sozlesmeye uymak zorunda, sozlesmeyi belirlemek icin degil.

"Inversion" kelimesi neden? DIP'siz haliyle high-level somut low-level'a bagimliydi. DIP ile ikisi de soyutlamaya bagimli oldu — bagimlilik oku **concrete'e dogru'dan, soyutlamaya dogru'ya cevriildi**.

Bes prensibin besini birden gozle gorulur sekilde isleten baska bir pattern bulmasi zor. Strategy bunu yapar. Mulakatciya "Strategy SOLID'in laboratuvaridir" cumlesi ezberden degil, deneyimden geliyor gibi durur.

Ne zaman KULLANMA — kidem ayiricisi

Junior hatasi her if/else'i Strategy'ye cevirmektir. Kod 5 dosyaya bolunur, hicbir esneklik kazanilmaz, sadece okuma zorlasir. Strategy'yi su durumlarda **kullanmaman** dogru karardir:

Ihtimaller az ve sabit. "Erkek/kadin" gibi iki branch'lik bir karar icin strategy yazmak overkill. Ucuncu ihtimal gelme olasiligi *gercekten* varsa strategy'ye gec.

Mantiklar tek satir. Her branch tek satirlik bir hesapsa, 4 dosya 40 satira cikarmak hastaligi azaltmaz, cogaltir.

Branch'ler birbirinin verisini paylasiyor. Yillik izin hesabinda mazeret iznine bakman gerekiyorsa, bunlari izole strategy yapmak zorlama olur. Belki bunlar ayri kurallar degil, tek bir kuralin varyantlari.

Secim kriteri yok. Strategy'nin calismasi icin "su durumda sunu kullan" diye bir karar var olmalı. Yoksa pattern'i niye uyguladigini sen de bilmiyorsun demektir.

Pratik heuristic: 3+ branch + her biri en az 5-10 satir + yeni branch gelme ihtimali yuksek → Strategy dusun. Iki sabit branch + iki satir → if/else kalsin.

Modern hali: Lambda'larla Strategy

Java 8 sonrası strategy cogu zaman ayri sinif gerektirmez. IzinOnayStrategy aslinda BiFunction<Izin, Personel, OnayResult> ile ayni kus. Stateless ve basit strategy'ler icin:

```
Map<IzinTipi, BiFunction<Izin, Personel, OnayResult>> strategies = Map.of(
    IzinTipi.DOGUM, (izin, p) -> new OnayResult(OnayDurum.OTOMATIK, 112),
    IzinTipi.RAPORLU, (izin, p) -> new OnayResult(OnayDurum.OTOMATIK, izin.getGunSayisi())
);
```

Bu da strategy. **Pattern'in ozu algoritmayi parametre olarak gecirmektedir**, syntactic sekli sinif mi lambda mi onemli degil. GoF 1994'te lambda yoktu, o yuzden kitap sinif-tabanlı anlatir.

Sen zaten her gun strategy kullaniyorsun farkinda olmadan:

`Collections.sort(list, comparator)` — Comparator bir strategy

`stream.filter(predicate)` — Predicate bir strategy

`Runnable r = () -> ...` — Runnable, Thread'in nasıl çalışacağını söyleyen strategy

`java.util.function` paketinin tamamı, küçük ve özelleşmiş strategy interface'lerinden oluşur.

ARGUS'tan strategy'lesebilir noktalar

VerexlCommand'taki excel_tipi switch'i — her excel tipinin parse/validate kuralı bir ExcellImportStrategy olabilir. excel_tipi=4 (EggtopCommand) ayrı strategy, excel_tipi=2 ayrı strategy.

PdfIzinKarti ve zamanDamgalalzinFormu — landscape/portrait, kart-tipi/form-tipi her biri bir PdfRenderStrategy. PDF generation'in iskelet kısmı (header, footer, page setup) Context'te kalır, içerik render strategy'de.

WfizinCommand'in tarihKontrolWfizin ve tarihKontrollznper kontrolleri — şirket bazlı muafiyet kuralı (Tosyali yarım gün) aslında TatilKontrolStrategy ile çözülebilirdi. Su an guard'lar inline; ileride başka şirket başka muafiyet ister, yine if (şirket == "X") patlaması başlar.

Mulakatta altin cumle: "Bulk upload sisteminde excel tipi basina farkli validation gerekiyordu, basta switch yigini vardi, strategy'ye cevirince yeni tip eklemek tek sinifa dustu ve mevcut kodu hic bozmadik." Pattern'i sadece bildigini degil, **aciyi yasayip cozduğunu** gosterir. Senior testinin orta yerinden geciyor.

Mulakat sorulari — derin halleriyle

Strategy ile State pattern farki?

Yapi ayni, niyet farkli. **Strategy'de** algoritmayi disaridan client secer — siralama yontemi, odeme tipi, izin onay kurali. Strategy'ler birbirinden habersizdir. **State'de** nesnenin kendi ic durumu degisince davranisi degisir — siparis "hazirlaniyor"dan "kargoda"ya gecer, gecisi state kendi yapar. State'ler birbirinden haberdardir, siklikla sonraki state'i kendileri belirler. *Tek cumleye:* Strategy'de disaridan kim oldugun belli, State'de iceriden kim oldugun degisior.

Strategy ile Template Method farki?

Template Method **inheritance** kullanir, Strategy **composition**. Template'da soyut bir base class iskeleti tanimlar, alt siniflar bosluklari doldurur. Strategy'de iskelet yok, **tum algoritma** stratejide. "Composition over inheritance" prensibi geregi genelde Strategy tercih edilir — daha esnek, daha test edilebilir.

Strategy'ler stateful olabilir mi?

Olabilir ama tehlikelidir. Spring'de singleton scope'lu strategy bean'in state tutarsa **thread safety** problemi cikar. Stateless tutmak en saglami. State gerekiyorsa metoda parametre olarak gecir, ya da @Scope("prototype") kullan.

Ayni interface'in 20 implementation'u olursa?

Ilk reaksiyon "harika, OCP isliyor". Ikinci reaksiyon kidemli reaksiyondur: "abartiya kactik mi?". 20 strategy yönetilmesi karmasiklasir. Bunlari **gruplamak**, hiyerarsik strategy yapmak, ya da bir kismini **Chain of Responsibility**'ye cevirmek daha dogrudur. Pattern tek basina hedef degil — karmasikligi yönetme aracidir, kendi karmasikligini yaratmaya baslarsa amaca aykiri duser.

Strategy secim mantigi nereye gider?

Uc secenek: (1) Client kendi secer, instance verir. (2) Context icinde Map ile cozulur — bizim ornek. (3) Ayri bir Factory yazarsin, sadece strategy secimini barindirir. Strategy cogaldikca 3. secenek temizdir.

Spring'de strategy'leri otomatik nasil topluyorsun?

Constructor'a List<Strategy> veya Map<String, Strategy> koy, Spring tum @Component'leri enjekte eder. Daha temizi: kendi Map<IzinTipi, ...>'ini constructor'da kur, her strategy desteklediğiTip() dondursun — yukaridaki ornegi bu yuzden oyle yazdik.

Strategy yerine enum kullansak ne olur?

Java enum metot icerebilir, calisir, az sayida strategy varsa pratik bile. **Ama** enum'a Spring bean inject etmek zor (enum Spring bean degil), testte mock'lamak zor, yeni tip eklemek icin enum dosyasini acman gerek — OCP zayıflar. Hızlı ve basit, ama esnek degil. Trade-off'unu bil.

Strategy'yi en kotu kullanim ornegi?

İki branch için strategy yazmak. Tek karakter karsilastirmasi için strategy yazmak. Her if'i strategy'ye cevirmek. Buna "pattern fetisizmi" denir, mulakatci bunu gormekten hoslanmaz. "Once gercek bir acim vardi, sonra pattern uyguladim" diyebilmek lazim — tersine degil.

2. Singleton Pattern

Bir sınıftan sadece bir tane instance olmasını dilden değil, sınıfın kendisinden garantile.

Once sezgi, sonra pattern

Su sahneyi al: büyük bir uçak gemisinde kaptan koprusu. O koprunun sadece **bir tane** olması gerekiyor. İki kaptan koprusu olsa, biri "sola don" derken diğeri "saga don" der, gemi delirir. Bazi seyler tek olmak zorunda — siyaseten değil, **mantiken**.

Yazılımda da aynı: bazı kaynaklar/servisler dogasi gereği tek olmalı. Aynı DB'ye birden fazla connection pool, aynı log dosyasına birden fazla logger, aynı parametre tablosunu birden fazla manager tasirsa **tutarsizlik + israf** çıkar.

Singleton: "Bu sınıfın bütün hayatı boyunca tek bir instance ile temsil edilecek, hiç kimse ikincisini yapamayacak."

Aciyi kodda gorelim

ARGUS'ta TSIRPRM tablosunda sistem parametreleri var — izin kurallari, ekran ayarlari, e-posta SMTP config'i. Bu parametreleri tutan bir KonfigurasyonYoneticisi sınıfın var. Naive yaklasim:

```
public class KonfigurasyonYoneticisi {
    private Map<String, String> parametreler;

    public KonfigurasyonYoneticisi() {
        // DB'ye git, TSIRPRM'den tum parametreleri cek
        // ~3 saniye suren agir bir islem
    }

    public String getParametre(String key) {
        return parametreler.get(key);
    }
}

// Her command'de:
public class WfizinCommand {
    public void execute() {
        KonfigurasyonYoneticisi km = new KonfigurasyonYoneticisi(); // 3 sn
        String paramX = km.getParametre("IZIN_MAX_GUN");
        // ...
    }
}
```

Sorun ne?

Her command 3 saniye bekliyor, kullanıcı ekrana giremiyor.

Memory'de 50 kopya dolasiyor — her command kendi map'ini tasiyor.

Tutarsizlik riski: admin parametreyi degistirdi, bazı command'lerin map'i eski parametre ile dolu.

DB her seferinde somuruluyor, network ve DB load'u bosuna.

Cozum sezgisi: Bu sinifin **bir tane** olsun, herkes ona ulassin, her bilgi tek yerde tutulsun.

Pattern'in iskeleti — adım adım

```
public class KonfigurasyonYoneticisi {

    // 1. Tek instance'i static field'da tut
    private static KonfigurasyonYoneticisi instance;

    private Map<String, String> parametreler;

    // 2. Constructor'i PRIVATE yap — disaridan kimse new yapmasin
    private KonfigurasyonYoneticisi() {
        this.parametreler = TSIRPRMdenCek();
    }

    // 3. Instance'a erisim kapisi — tek giris noktası
    public static KonfigurasyonYoneticisi getInstance() {
        if (instance == null) {
            instance = new KonfigurasyonYoneticisi();
        }
        return instance;
    }

    public String getParametre(String key) {
        return parametreler.get(key);
    }
}
```

Uc temel bilesen:

private static instance field — Tek nussha burada saklanir.

private constructor — Disaridan new yapilmasi fiziksel olarak imkansiz.

public static getInstance() — Tek mesru giris noktası.

Kullanım:

```
String param = KonfigurasyonYoneticisi.getInstance().getParametre("IZIN_MAX_GUN");
```

new kelimesi hic yok. Sinif kendisi nussha sayisini kontrol ediyor.

Yukaridaki kod neden BOZUK — thread safety

Yukaridaki naive singleton bir bombanın ustunde oturuyor. Coklu thread senaryosu:

```
Thread A: getInstance() cagirdi
Thread A: instance == null? Evet

Thread B: getInstance() cagirdi
Thread B: instance == null? EVET (A henuz yapmadi)

Thread A: new KonfigurasyonYoneticisi()
Thread B: new KonfigurasyonYoneticisi()
```

İki tane instance uretildi. Singleton'un tum amaci bosa. Multi-thread ortamda (her web request bir thread, ARGUS'ta surekli boyle) bu **mutlaka** olur.

Thread-safe singleton — 5 farkli yaklasim

1. Eager initialization (en basit)

```
public class K {  
    private static final K INSTANCE = new K();  
    private K() {}  
    public static K getInstance() { return INSTANCE; }  
}
```

Class load aninda olusur, thread-safe (JVM garantisi). **Eksisi:** hic kullanilmasa bile bellekte yer kaplar. Sinif agirsa bosa yer acar.

2. Synchronized method

```
public static synchronized K getInstance() {  
    if (instance == null) instance = new K();  
    return instance;  
}
```

Calisir ama yavas — her cagrida lock alir, oysa lock sadece ilk cagrida gerekli. Yuksek trafikte performans sorunu.

3. Double-Checked Locking (akilli cozum)

```
public class K {  
    private static volatile K instance; // volatile KRITIK  
  
    private K() {}  
  
    public static K getInstance() {  
        if (instance == null) { // birinci check (lock'suz, hizli)  
            synchronized (K.class) {  
                if (instance == null) { // ikinci check (lock'lu, kesin)  
                    instance = new K();  
                }  
            }  
        }  
        return instance;  
    }  
}
```

Lock'u sadece instance null'ken aliyor, sonraki tum cagrilar lock'suz akiyor.

volatile neden lazim? instance = new K() aslinda atomic degil — JVM bunu uc adimda yapar: (1) bellek ayir, (2) referansi ata, (3) constructor'i calistir. JVM optimizasyonu 2 ile 3'un sirasini degistirebilir. Bu durumda baska bir thread *instance != null* gorur ama yari olusmus objeye dokunur — patlar. **volatile** happens-before garantisi vererek bu yeniden siralamayi engeller.

4. Bill Pugh / Initialization-on-demand holder (en sik)

```
public class K {
    private K() {}

    private static class Holder {
        private static final K INSTANCE = new K();
    }

    public static K getInstance() {
        return Holder.INSTANCE;
    }
}
```

Holder class'i **ilk getInstance cagrisina kadar yuklenmez**. JVM class loading'in thread-safe olmasini garanti eder. Lazy + thread-safe + lock'suz. Joshua Bloch'un cok begendigi yontemlerden.

5. Enum singleton (en guvenli)

```
public enum K {
    INSTANCE;

    public void doSomething() { ... }
}
```

Kullanım: `K.INSTANCE.doSomething()`. Reflection, serialization, cloning'e karsi **bagisik** — JVM enum'larda bunlari zaten engeller. Joshua Bloch "Effective Java"da bunu oneriyor. **Eksisi:** enum hissi vermesi, dependency injection'in enum'lara kotu uymasi.

Singleton'un kara yuzu — "anti-pattern" iddiası

Bazi yazilim yazarlari Singleton'a "anti-pattern" der. Neden?

Global state. Singleton aslinda cakal bir global degisken. Her yerden erisilebilir, izlemesi zor. Bir gun KonfigurasyonYoneticisi.getInstance().setParametre(...) diye bir cagri bir yere sizar, tum sistem etkilenir, sebep bulamazsin.

Hidden dependency. `IzinService.onayla()` metodunun imzasina bakarak KonfigurasyonYoneticisi'ye bagimli oldugunu anlayamazsin. Iceride sakli getInstance() cagrisi var. Bu test yazmayi, debug etmeyi, dokumante etmeyi zorlastirir.

Test edilebilirlik dusuklugu. Mock'lamak zordur — constructor private, instance static. PowerMock gibi siyah buyu kutuphanelerine basvurmak gerekir.

Siki kuplaj. Singleton'a bagimli siniflar onsuz calisamaz. Test ortaminda onu degistirip basit bir mock veremezsin.

Sinif birden fazla sorumluluk tasir. Hem is mantigini yapmak hem de tekligini yönetmek ayni sinifa dusuyor — Single Responsibility ihlali.

Modern dünyada Singleton: Spring'in çözümü

Anti-pattern eleştirilerinin **tamami**, sen elle Singleton yazdığında geçerli. Spring'in @Component / @Service / @Repository bean'leri default olarak singleton scope'lu — yani container "bu sınıftan tek instance yapıyorum, sana onu veriyorum" diyor. **Ama** problemlerin hicbiri yok çünkü:

Constructor injection ile bağımlılık **gorunur** (hidden değil).

Test'te container kullanmadan, ilgili sınıfı normal new ile oluşturup mock'lar inject edebilirsiniz.

Sınıf kendisi tekliğini yönetmiyor, **container yönetiyor** — Single Responsibility korunuyor.

Birden fazla ApplicationContext varsa her birinde ayrı bir instance var — yani gerçek bir test izolasyonu mümkün.

```
@Service
public class KonfigurasyonYoneticisi {
    private final ParametreRepository repo;

    // Spring tek instance üretir, repo'yu inject eder
    public KonfigurasyonYoneticisi(ParametreRepository repo) {
        this.repo = repo;
    }

    @PostConstruct
    public void warmup() {
        // TSIRPRM'i belleğe yükle
    }
}
```

Modern Java'da elle Singleton yazmak çoğu zaman gereksizdir — DI container'ın isidir. Sen "bunun tek instance'i olsun" diye düşündüğün anda, "ben Spring bean'i yapıyorum" demektir, fazlası değil.

SOLID ile bağlantısı (Singleton burada zayıf!)

Kritik bir nokta: **Singleton SOLID ile dostane bir pattern değildir**. Aksine, sıklıkla ihlal etmesiyle bilinir:

SRP ihlali: sınıf hem işini yapıyor hem teklik yönetiyor.

DIP ihlali: KonfigurasyonYoneticisi.getInstance() çağırان herkes **somut sınıfa** bağımlı — interface yok, soyutlama yok.

OCP zayıflığı: singleton'u değiştirmek/değiştirilmek istemediği için sistem üzerinde kemiklesmiş bir nokta olur.

Mulakat altın cumlesi: "Singleton tek başına yazıldığında SOLID'e aykırı eğilimlidir, çünkü hidden global state üretir. Modern uygulamada bunu DI container'a delege ederek SOLID'i koruruz." Bu cevap çok kıdemli durur.

Mulakat sorulari — derin halleriyle

Singleton nasıl kirilir?

Üç yol: (1) **Reflection** — `Constructor.setAccessible(true)` ile private constructor erişilir hale gelir, ikinci instance üretilir. (2) **Serialization** — singleton serializable ise, deserialize edildiğinde yeni instance olur. Korunma: `readResolve()` metodu yazıp mevcut instance'i dondurmek. (3) **Cloning** — `Cloneable` implement ediyorsa `clone()` ikinci instance üretir. Korunma: `clone()` override edip exception fırlatmak.

Enum singleton bunların hiçbirine açık değildir — JVM korur. O yüzden Bloch onu önerir.

Singleton vs Static class farkı?

Singleton bir **nesnedir** — interface implement edebilir, kalıtım alabilir, parametre olarak geçirilebilir, polimorfik kullanılabilir, lazy yüklenebilir. Static class sadece statik metodların tutucusudur — OOP özelliklerinden yararlanamaz, dependency injection'a girmez.

Spring'in singleton'i GoF singleton mu?

Hayır, farklı. GoF singleton classloader başına bir instance. Spring singleton `ApplicationContext` başına bir instance. Aynı JVM'de iki context çalışsa (mesela bir entegrasyon test'inde), iki ayrı Spring singleton olur — GoF singleton'da bu olmaz.

volatile olmadan double-checked locking çalışır mı?

Java 5'ten önce çalışmazdı — memory model garanti vermiyordu. Java 5+'da volatile ile çalışır. Volatile yoksa instruction reordering yüzünden başka thread yarı oluşmuş objeye erişebilir.

Singleton thread-safe ama içindeki state thread-safe değilse?

Singleton'lık **nesnenin tekliğini** garanti eder, içindeki field'ların concurrent erişimini değil. State tutuyorsa `ConcurrentHashMap`, atomic değişken veya senkronizasyon kullanman gerekir.

Modern Java'da hala Singleton yazmalı mıyız?

Cogu zaman hayır — DI container scope'una bırak. Yazıyorsan tercihen enum veya Bill Pugh holder kullan, naive lazy init'ten kaçın.

3. Proxy Pattern

Bir nesneye dogrudan erisimi engelleyip araya baska davranislari yerlestirmek icin, ayni arayuze sahip bir vekil koy.

Once sezgi, sonra pattern

Su sahneye bak: CEO'nun ofisinin onunde bir sekreter oturuyor. CEO ile konusmak isteyen herkes once sekreterden geciyor. Sekreter:

Randevun var mi bakiyor (**yetki kontrolu**)

"CEO toplantida, sekreterden bilgi alabilir misin?" diyor (**cache, lazy**)

"Su kisi su saatte geldi" diye not aliyor (**log**)

CEO odadayken kapiyi calip seni iceri aliyor (**proxy → real**)

CEO'nun kendisi degismedi. Onunla ayni isi yapan biri yok aslinda — sekreter sadece **erisimi kontrol ediyor**. Disaridan bakan kisi sekreteri "CEO ile iletisim noktası" olarak goruyor ve farkinda bile degil ki gercek CEO arkada.

Proxy pattern tam bu: **gercek nesneyle ayni interface'i sunan, ama araya kendi mantigini sokan bir araci**.

Neden ihtiyac dogdu? — bes aci, bir cozum

Aci 1: Pahali nesne, lazy yuklensin. Bir personel kaydini DB'den cektiginde, onun 20 yillik izin gecmisini de mi yuklemek gerekiyor? Hayir, ihtiyac olunca. Ama kod tarafinda `personel.getIzinler().size()` dogal gorunmeli.

Aci 2: Yetki kontrolu. ARGUS REST servislerinde her endpoint icin JWT auth kontrol etmen lazim. Bunu her metodun basina yazmak istemiyorsun — kod kirleniyor, unutma riski var.

Aci 3: Logging / Audit. `AcilDurumService.kaydet()` cagrildiginda "kim cagirdi, ne zaman, hangi parametrelerle" loglamak istiyorsun. Metoda yazsan is mantigini kirletir.

Aci 4: Caching. `getParametre("MAX_IZIN_GUN")` ayni sonuc icin 100 kez cagriliyor. Her seferinde DB'ye gitmek aptallik.

Aci 5: Uzaktaki nesne. Baska bir sunucudaki servisi yerel mis gibi kullanmak istiyorsun. RMI, gRPC, REST client'lar — hepsi proxy.

Tum bu acilarin cozumu: araya **ayni arayuzu sunan bir vekil** koymak.

Kodda gorelim — protection proxy

Senin `AcilDurumService`'ini ele alalim. Once gercek hali:

```
public interface AcilDurumService {  
    void kaydet(AcilDurumRequest req);  
    AcilDurum getir(Long id);  
}
```

```

}

public class AcilDurumServiceImpl implements AcilDurumService {
    private final AcilDurumRepository repo;

    public void kaydet(AcilDurumRequest req) {
        repo.save(...); // gercek is
    }

    public AcilDurum getir(Long id) {
        return repo.findById(id); // gercek is
    }
}

```

Simdi yetki kontrolu gerekti. Service'in icine if (hasRole(...)) yazmıyoruz, proxy yazıyoruz:

```

public class AcilDurumServiceSecurityProxy implements AcilDurumService {

    private final AcilDurumService gercek;          // gercek nesne
    private final KullaniciContext context;

    public AcilDurumServiceSecurityProxy(AcilDurumService gercek, KullaniciContext context) {
        this.gercek = gercek;
        this.context = context;
    }

    @Override
    public void kaydet(AcilDurumRequest req) {
        if (!context.hasRole("HR_ADMIN")) {
            throw new YetkisizException("Acil durum kaydi icin yetkin yok");
        }
        gercek.kaydet(req); // gercege delege et
    }

    @Override
    public AcilDurum getir(Long id) {
        // okuma serbest, direkt gercege gonder
        return gercek.getir(id);
    }
}

```

Client tarafı:

```

AcilDurumService service = new AcilDurumServiceSecurityProxy(
    new AcilDurumServiceImpl(repo),
    context
);

service.kaydet(req); // once yetki, sonra kayit

```

Client AcilDurumService interface'iyle calisiyor — **proxy mi gercek mi oldugunu bilmiyor**. Iste guzelligi bu. Yarın proxy'yi cikarsan tek satir kod degismez.

Pattern'in iskeleti — uc oyuncu

Subject (Interface) — Gercek ve proxy'nin uyduğu sözleşme. AcilDurumService.

RealSubject — Asıl işi yapan. AcilDurumServiceImpl.

Proxy — Aynı interface'i implement eder, içinde RealSubject'e referans tutar, çağrılarını kendi mantığını ekleyerek delege eder.

Kritik: client her zaman Subject interface'yle konusur. Proxy hangi davranisi ekliyorsa eklesin, sozlesmeyi bozmuyor.

Proxy turleri — her biri bir aciya cevap

Virtual Proxy (lazy loading proxy)

Nesne pahali, ihtiyac olana kadar gercegi olusturma. Hibernate'in @OneToMany iliskilerinde tam olarak bu olur. Sen `personel.getIzinler()` dediginde gelen aslinda `List<Izin>` degil, **PersistentBag** adinda bir proxy. Ilk gercek kullanimda (`.size()`, `.iterator()`, `.get(0)`) Hibernate DB'ye gider, gercek listeyi yukler. Session kapaliysa **LazyInitializationException** patlar — bu mulakatin klasigi.

Protection Proxy (yetki kontrolu)

Yukaridaki `AcilDurumServiceSecurityProxy` ornegi. Spring Security'nin `@PreAuthorize`'i arka planda bunu yapar.

Remote Proxy

Uzaktaki bir nesneyi yerel gibi kullanma. RMI stub'lari, gRPC client'lari, Feign client'lari — hepsi remote proxy. Sen `userClient.getUser(123)` diyorsun, proxy HTTP request atip cevabi deserialize ediyor.

Caching Proxy

Spring'in `@Cacheable`'i. Ilk cagrida gercek metoda gider, sonucu cache'ler. Ikinci ve sonraki cagrilarda cache'ten doner — gercek metod hic tetiklenmez.

Logging / Audit Proxy

Her cagridan once/sonra log atar. Cogru zaman aspect olarak yazilir ama altinda proxy yatar.

Spring'in buyuk sirri: AOP = Proxy

Asagidaki Java kodun:

```
@Service
public class IzinService {

    @Transactional
    public void izinKaydet(Izin izin) {
        // ...
    }

    @Cacheable("personel-izin")
    public List<Izin> personelIzinleri(Long personelId) {
        // ...
    }
}
```

Sen `@Autowired IzinService` dediginde Spring sana **gercek IzinService'i vermiyor**. Senin icin bir proxy yaratmis, onu veriyor. O proxy `izinKaydet` cagrildiginda:

1. Transaction baslatir

2. Gercek metoda delege eder
3. Hata yoksa commit, varsa rollback
4. Senin yazmadigin butun bu logic etrafini sarar

@Cacheable icin de: cache'te key var mi bakar, varsa direkt doner; yoksa gercek metoda gider, sonucu cache'ler.

Spring proxy'leri iki sekilde uretir

JDK Dynamic Proxy — Sinif bir interface implement ediyorsa. Runtime'da Proxy.newProxyInstance ile yeni bir sinif uretir. Bu sinif interface'i implement eder ve her metod cagrisini bir InvocationHandler'a yonlendirir.

CGLIB Proxy — Interface yoksa, sinifi dogrudan **subclass'lar**. Metodlari override eder. Bu yuzden final siniflar/metodlar proxy'lenemez. Spring 4.x+ default'u CGLIB cunku kullanci interface cikarmak zorunda kalmiyor.

Self-invocation tuzagi — mulakatin favorisi

```
@Service
public class IzinService {

    @Transactional
    public void izinKaydet(Izin izin) {
        validate(izin);
        this.persistEt(izin);    // <-- BU SATIRA DIKKAT
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void persistEt(Izin izin) {
        // ...
    }
}
```

Soru: persistEt'in REQUIRES_NEW annotation'i calisir mi?

Cevap: HAYIR, calismaz. Cunku proxy **disaridan** bir sarmalama. izinKaydet zaten proxy uzerinden cagrildi, iceri girdi (gercek metod). Sonra this.persistEt() dediginde, this proxy degil, **gercek nesne**. Cagri dogrudan gercek metoda gider, proxy aradan cikar, REQUIRES_NEW annotation'i gormezden gelinir.

Cozumler:

AopContext.currentProxy() ile proxy'yi tekrar al, ondan cagir

Self-injection (@Autowired IzinService self; — Spring proxy'yi inject eder)

Metodu baska servise tasi

Proxy ile karistirilan iki pattern

Proxy vs Decorator

Ikisi de ayni interface'i implement edip araya girer. **Yapisal olarak ayirt etmek zor**, niyet uzerinden ayrilir:

Proxy: erisimi **kontrol eder** (guvenlik, lazy, remote, cache). Genelde tek katmandir.

Decorator: davranisi **genisletir, ekler** (yeni ozellik). Siklikla chain'lenir — `BufferedReader(InputStreamReader(FileInputStream(...)))`.

Proxy vs Adapter

Adapter: uyumsuz arayuzleri uyumlu hale getirir. Eski API'yi yeni API'ye ceviris. Interface'i degistirir.

Proxy: ayni arayuzu korur, sadece araya girer.

Yani Adapter "yabanci dile cevirmen", Decorator "ekstra ozellikli", Proxy ise "kapici / sekreter".

SOLID ile baglantisi

Strategy kadar yogun degil ama var:

OCP — Mevcut sinifi (RealSubject) degistirmeden davranis ekleyebilirsiniz. Proxy yazmak Service'in koduna dokunmaz. Yetki kontrolu eklemek icin AcilDurumServiceImpl acilmadi.

SRP — Asil is Impl'de, cross-cutting concerns (yetki, log, cache) proxy'de. Siniflar tek sorumluluk tasiyor.

DIP — Client interface ile calisir, proxy mi gercek mi oldugunu bilmez. Bagimlilik soyutlamaya, somut sinifa degil.

Ne zaman Proxy yazilir, ne zaman yazilmaz?

Yazilir: cross-cutting concerns var (auth, log, cache, transaction), lazy loading lazim, remote'a erisim var, gercek nesneye erisimi kontrol etmek gerekiyor.

Yazilmaz: tek bir yerde tek bir cross-cutting need varsa, proxy overkill. Direkt metod icine yaz, gec. Proxy reusable bir endiseyi soyutlamak icindir — tek seferlik icin zahmetine degmez.

Modern dunyada: sen elle proxy yazmaktan cok, Spring AOP veya aspect kullanarak proxy olusturursun. @Around, @Before, @After ile cross-cutting concern'leri annotate edersin, Spring sana proxy uretir.

Mulakat sorulari — derin halleriyle

JDK Dynamic Proxy vs CGLIB?

JDK proxy interface gerektirir, performansi hafif daha yavas ama Java'nin yerli ozelligi — ekstra kutuphane yok. CGLIB subclass yapar, interface gerektirmez, final ile calismaz, ASM kutuphanesi kullanir. Spring 4+ default'u CGLIB cunku interface zorunlulugu pek cok ekibe yuktur.

Hibernate lazy loading nasıl proxy kullanır?

Hibernate LazyInitializer üzerinden bir proxy nesnesi üretir. İlk gerçek erişimde (getter çağırısı, .size(), iterator) initialize olur ve DB'ye gider. Session kapanmısa LazyInitializationException. Çözüm: session'i açık tut (open-session-in-view, transaction içinde don), JOIN FETCH ile eager çek, ya da DTO mapper kullan.

Spring AOP'da neden self-invocation çalışmaz?

Yukarıda detaylı anlattık — `this.method()` proxy'yi bypass eder, çünkü `this` gerçek nesneyi gösteriyor.

Proxy pattern'i elle yazsan static mi dynamic mi olur?

Yukarıdaki `AcilDurumServiceSecurityProxy` **static proxy** — derleme zamanında biliniyor, elle yazılmış. Spring/Hibernate runtime'da class üretir — **dynamic proxy**.

Spring'de @Transactional neden public metodlarda çalışıyor, private'larda çalışmıyor?

Çünkü CGLIB proxy subclass yapar, override edebilmek için metod public veya protected olmalı, private metodlar zaten override edilemez. JDK dynamic proxy ise sadece interface metodlarını proxy'ler, interface metodları zorunlu public.

DI ve proxy ilişkisi?

Spring DI yaparken sana ciplak bean'i değil, **proxy'yi** inject eder (eger AOP/@Transactional/@Cacheable varsa). Bu yüzden bazen `getClass()` baktığında `$$EnhancerByCGLIB$$` gibi garip isimler görürsün — proxy göstergesi.

Bir sınıf hem proxy hem decorator olabilir mi?

Teknik olarak evet — aynı yapı. Ama niyet karışıklığı yaratır, kod okuyan anlamaz. Birini sec. Sınıf adı bunu yansıtsin: `LoggingProxy` mi `CachingDecorator` mi.

Son Sayfa — Mulakat Cheat Sheet

Tek cumlede her pattern

Strategy: ayni isin farkli yapilma sekillerini ayri siniflara izole et, runtime'da seg.

Singleton: sinifin tek instance'li olmasini sinifin kendisine garantilet.

Proxy: gercek nesneye dogrudan erisim yerine, ayni arayuze sahip vekil uzerinden eris.

SOLID dagilimi

Strategy: SOLID'in laboratuvari. Bes prensibin besine de saf ornek. OCP'nin kanonik temsilcisi.

Singleton: SOLID'e karsi temayullu. Naive halinde SRP/DIP/OCP ihlali. Cozum: DI container scope'una birak.

Proxy: OCP (kodu degistirmeden davranis ekleme), SRP (cross-cutting'i ayirma), DIP (interface uzerinden).

"Acim vardi, pattern uyguladim" cumlelerin

Strategy: "Bulk upload'da excel tipi basina farkli validation gerekiyordu, switch yigini vardi, strategy'ye cevirince yeni tip eklemek tek sinifa dustu ve mevcut kodu hic bozmadik."

Singleton: "Konfigurasyon yoneticisini her command'de new'lemek 3 saniye sururdu, Spring bean'i yapinca tek instance oldu, hem hizlanma hem tutarli durum kazandik."

Proxy: "REST endpoint'lerinde her metoda yetki check'i yazmak yerine Spring Security'nin @PreAuthorize'ini kullandik, proxy arka planda calisip is mantigini temiz birakti."

Mulakat tuzaklari — sik sorulan favoriler

Strategy vs State — nereye karar veriyor: client mi, kendi ic durum mu?

Strategy vs Template Method — composition mu, inheritance mi?

Singleton'i kirma yollari — reflection, serialization, cloning.

Singleton thread safety — eager / synchronized / DCL / Bill Pugh / enum.

Proxy vs Decorator — kontrol mu, genisletme mi?

Spring proxy self-invocation problemi — this. cagrisi proxy'yi bypass eder.

JDK Dynamic Proxy vs CGLIB — interface var/yok ayrimi.

Son taktik: her pattern icin "ne zaman KULLANMA" cumlesini ezberle. Bu cumle seni junior'dan ayirir. "X pattern'i her seyin cozumü degil; su durumda overkill olur, su durumda gercekten gerekli" diyebilen bir mulakat adayi, mulakatcinin gozunde 1 seviye yukseliyor. Bol sans Batur.